

PRICING AND HEDGING EUROPEAN OPTIONS

**Three Numerical Methods, Their Cost, and What Discrete Hedging Really
Leaves Behind**

October 2026



Stéphane N'Da

github.com/StephaneMian/bs-pricing-hedging

Independent research project

EXECUTIVE SUMMARY

The Black–Scholes model gives a closed-form price for a European option, which makes it an ideal test bench: every numerical method can be checked against an exact answer, and every error can be measured rather than guessed. This project uses that property to answer two practical questions.

The first is a question of numerical method. The same call option is priced by Monte Carlo simulation (plain, antithetic variates, control variate), by a finite-difference solution of the pricing PDE, and in closed form. For a one-dimensional problem the PDE is far more efficient: a Crank–Nicolson grid reaches an error of 4×10^{-5} in about 70 ms, whereas two million Monte Carlo paths leave an error of about 10^{-2} in comparable time. The measured order of convergence of the PDE scheme is 2.00. Variance reduction matters, but its value depends on the contract: the control variate divides the variance by 50 for a deep in-the-money call and only by 2 for an out-of-the-money one. A second finding concerns Greeks rather than prices. Plain Crank–Nicolson produces a price that is correct to second order but a gamma that oscillates near the strike, with a maximum error of 10.7 against a true value of 0.06. Two implicit start-up steps (Rannacher smoothing) bring that error down to 2×10^{-4} .

The second question is what a trader keeps when hedging in discrete time. A short call hedged N times over one year leaves a P&L whose standard deviation decreases as $N^{-0.489}$ in simulation, close to the theoretical $N^{-1/2}$ and within 8% of the Derman–Kamal approximation at every frequency tested. When the option is sold at 20% implied volatility but the market realises 25%, the average loss equals the price difference (-1.93 for a premium of 9.41) whichever volatility is used for the hedge ratio. What changes is the dispersion: hedging with the realised volatility halves the standard deviation of the P&L (0.53 against 0.98), and the remaining dispersion is almost exactly the discretisation error.

All results are reproducible from a single script with fixed seeds, and the pricing functions are covered by unit tests that compare every numerical method with its closed form.

CONTENTS

1	Setting and Methods	3
1.1	Model	3
1.2	Monte Carlo estimators	3
1.3	Finite differences	3
1.4	Monte Carlo Greeks	4
1.5	Discrete delta hedging	4
2	Results	5
2.1	Variance reduction depends on the contract	5
2.2	Accuracy against cost	5
2.3	Why Rannacher smoothing matters for Greeks	6
2.4	Monte Carlo deltas	7
2.5	Hedging error and rebalancing frequency	8
2.6	Selling volatility too cheaply	8
3	Limits and Extensions	10
	References	11
	Appendix: Key Functions	12

1

SETTING AND METHODS

1.1 MODEL

Under the risk-neutral measure the stock follows a geometric Brownian motion

$$dS_t = rS_t dt + \sigma S_t dW_t, \quad (1.1)$$

with constant rate r and volatility σ . The price of a European call with strike K and maturity T is

$$C = S_0 \Phi(d_1) - Ke^{-rT} \Phi(d_2), \quad d_{1,2} = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}. \quad (1.2)$$

Unless stated otherwise the experiments use $S_0 = 100$, $T = 1$, $r = 3\%$, $\sigma = 20\%$. The at-the-money call is then worth 9.4134.

1.2 MONTE CARLO ESTIMATORS

The terminal price is simulated exactly, $S_T = S_0 \exp((r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z)$ with $Z \sim \mathcal{N}(0, 1)$, so the only error is statistical. Three estimators are compared.

- **Plain.** The sample mean of $e^{-rT}(S_T - K)^+$.
- **Antithetic variates.** Each draw Z is paired with $-Z$ and the two payoffs are averaged. The standard error is computed on the pair averages, which are independent; computing it on the $2m$ individual payoffs would understate it.
- **Control variate.** The discounted terminal price $Y = e^{-rT}S_T$ has known mean S_0 . The estimator $X - \beta(Y - S_0)$ uses the sample-optimal $\beta = \text{Cov}(X, Y)/\text{Var}(Y)$. Estimating β on the same sample introduces a bias of order $1/n$, negligible here.

The efficiency of an estimator is measured by its variance per payoff evaluation, so that an antithetic pair counts as two evaluations.

1.3 FINITE DIFFERENCES

With $x = \ln S$ and $\tau = T - t$, the price $u(x, \tau)$ solves

$$\partial_\tau u = \frac{1}{2}\sigma^2 \partial_{xx} u + (r - \frac{1}{2}\sigma^2) \partial_x u - ru, \quad (1.3)$$

a constant-coefficient equation, which is why the log transform is used. The grid is centred on $\ln S_0$ and spans $\pm 5\sigma\sqrt{T}$; Dirichlet conditions use the asymptotic prices (0 and $S - Ke^{-r\tau}$ for a

call). Time stepping is Crank–Nicolson, second order in time and space, with a tridiagonal system solved at each step.

Crank–Nicolson is unconditionally stable but not strongly damping: high-frequency components of the initial condition are multiplied by a factor close to -1 at each step when $\Delta\tau/\Delta x^2$ is large. The kink of the payoff at the strike contains exactly such components. The Rannacher variant replaces the first two time steps by four fully implicit half-steps, which damp them before the Crank–Nicolson steps begin.

1.4 MONTE CARLO GREEKS

Delta is estimated in three ways: the pathwise derivative $e^{-rT} \mathbf{1}_{S_T > K} S_T / S_0$, which is valid because the call payoff is Lipschitz; the likelihood-ratio estimator $e^{-rT} (S_T - K)^+ Z / (S_0 \sigma \sqrt{T})$, which differentiates the density instead of the payoff; and a central bump-and-revalue with common random numbers and a 1% bump.

1.5 DISCRETE DELTA HEDGING

The seller of one call receives the premium computed with an implied volatility σ_{imp} , buys Δ shares computed with a hedging volatility σ_h , and rebalances at N equally spaced dates. The stock is simulated under the real-world measure with drift $\mu = 8\%$ and volatility σ_{real} . Cash earns r . The quantity studied is the discounted P&L at maturity, after paying the option payoff.

For small time steps, Derman and Kamal give the approximation

$$\text{sd}(\text{P\&L}) \approx \sqrt{\frac{\pi}{4}} \frac{\sigma \mathcal{V}}{\sqrt{N}}, \quad (1.4)$$

where $\mathcal{V}$ is the option vega. The $N^{-1/2}$ rate is the testable part of this statement.

2

RESULTS

2.1 VARIANCE REDUCTION DEPENDS ON THE CONTRACT

Table 2.1 reports, for one million payoff evaluations, the price, its standard error, the distance to the closed form in standard errors (z), and the variance ratio relative to plain Monte Carlo.

Strike	Estimator	Price	Std. error	z	Variance ratio
80 (ITM)	plain	23.2063	0.0189	-0.93	1.0
	antithetic	23.2329	0.0069	1.29	7.6
	control variate	23.2305	0.0027	2.41	49.5
	closed form	23.2240			
100 (ATM)	plain	9.4018	0.0141	-0.82	1.0
	antithetic	9.4281	0.0105	1.40	1.8
	control variate	9.4183	0.0058	0.85	5.9
	closed form	9.4134			
120 (OTM)	plain	2.7577	0.0080	-1.11	1.0
	antithetic	2.7765	0.0075	1.32	1.1
	control variate	2.7651	0.0055	-0.26	2.1
	closed form	2.7666			

TABLE 2.1: Monte Carlo estimators with 10^6 payoff evaluations each.

The pattern has a simple explanation. Both techniques exploit the correlation between the payoff and the underlying. Deep in the money, the call behaves almost like the forward $S_T - K$, which is linear in S_T : the control variate removes nearly all of its variance, and the antithetic pair cancels the linear part of the noise. Out of the money, most paths pay zero and the payoff is a strongly non-linear function of Z ; correlation with S_T is weak and both gains shrink. A variance ratio measured on one strike should therefore not be quoted as a property of the method.

The z -scores lie between -1.1 and 2.4 . The value 2.41 is the largest of nine, which is unremarkable for nine roughly normal draws, and the unit tests use a separate seed and a four-standard-error tolerance.

2.2 ACCURACY AGAINST COST

Figure 2.1 plots the absolute error against CPU time. For Monte Carlo the error is the root-mean-square error over 30 independent seeds at each sample size.

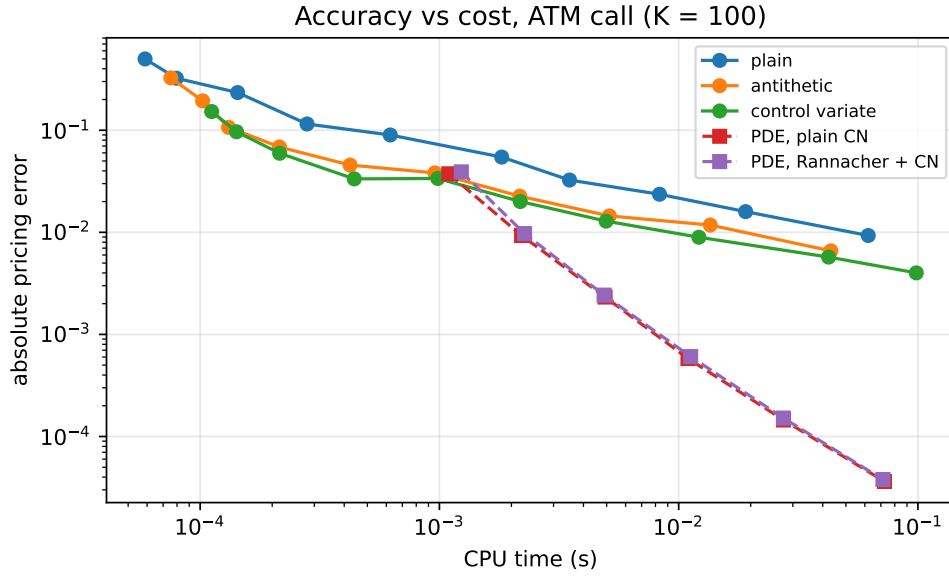


FIGURE 2.1: Error against CPU time for the at-the-money call.

The Monte Carlo curves have slope close to $-1/2$ in log–log scale, as expected from the central limit theorem, and variance reduction shifts them down without changing the slope. The PDE curve has a slope of about -1 in time: halving $\Delta\tau$ and Δx quadruples the work and divides the error by four. The fitted order of convergence in the number of time steps is 2.00. At 800 time steps and 1 600 space nodes the error is 3.6×10^{-5} in 69 ms, about 250 times smaller than plain Monte Carlo and 100 times smaller than the control variate at a similar cost.

This ranking is specific to one dimension. The cost of a grid grows exponentially with the number of state variables, while Monte Carlo does not care; for a basket of five assets or a path-dependent payoff the conclusion reverses.

2.3 WHY RANNACHER SMOOTHING MATTERS FOR GREEKS

For the price at S_0 , plain Crank–Nicolson and the Rannacher variant give practically the same error at every grid size tested (3.6×10^{-5} against 3.8×10^{-5} at the finest grid). The difference appears in the gamma, computed from the grid as $(\partial_{xx}u - \partial_x u)/S^2$. Figure 2.2 uses a short maturity ($T = 0.1$) and only ten time steps on a fine space grid, a combination that makes $\Delta\tau/\Delta x^2$ large.

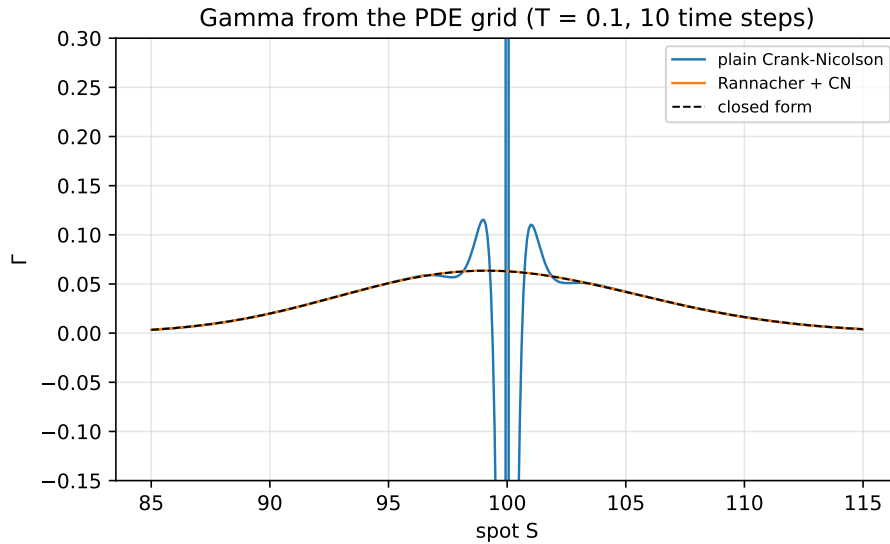


FIGURE 2.2: Gamma from the PDE grid. The plain scheme spikes to about 10 at the strike; the axis is clipped.

Near the strike the maximum error on gamma is 10.7 with plain Crank–Nicolson and 1.6×10^{-4} with Rannacher start-up, for a true gamma of about 0.06 . A pricing error of 10^{-4} can hide a Greek that is wrong by two orders of magnitude, and a hedger uses the Greek. Testing only the price would not have revealed the problem.

2.4 MONTE CARLO DELTAS

Table 2.2 gives the standard error of each delta estimator with 5×10^5 paths.

Strike	True delta	Pathwise	Likelihood ratio	Bump (CRN)
80	0.9140	5.4×10^{-4}	2.8×10^{-3}	5.3×10^{-4}
100	0.5987	8.3×10^{-4}	2.0×10^{-3}	8.2×10^{-4}
120	0.2541	7.3×10^{-4}	1.3×10^{-3}	7.3×10^{-4}

TABLE 2.2: Standard errors of three delta estimators (5×10^5 paths).

The pathwise estimator has between 3 and 27 times less variance than the likelihood ratio. The likelihood ratio multiplies the payoff by Z , which adds noise, most visibly in the money where the payoff is large. Its advantage is elsewhere: it does not require a differentiable payoff and remains usable for a digital option, where the pathwise derivative is zero almost everywhere. Bump-and-revalue with common random numbers matches the pathwise estimator, since for a small bump it computes nearly the same quantity, but it needs two full pricings per Greek.

The three estimators were run with the same seed, so their deviations from the true value are correlated: at each strike, all three land about two standard errors below the true delta. This is one draw of the random numbers and not a bias; the unit tests use an independent seed.

2.5 HEDGING ERROR AND REBALANCING FREQUENCY

Figure 2.3 shows the standard deviation of the hedging P&L over 50 000 paths when all three volatilities equal 20%.

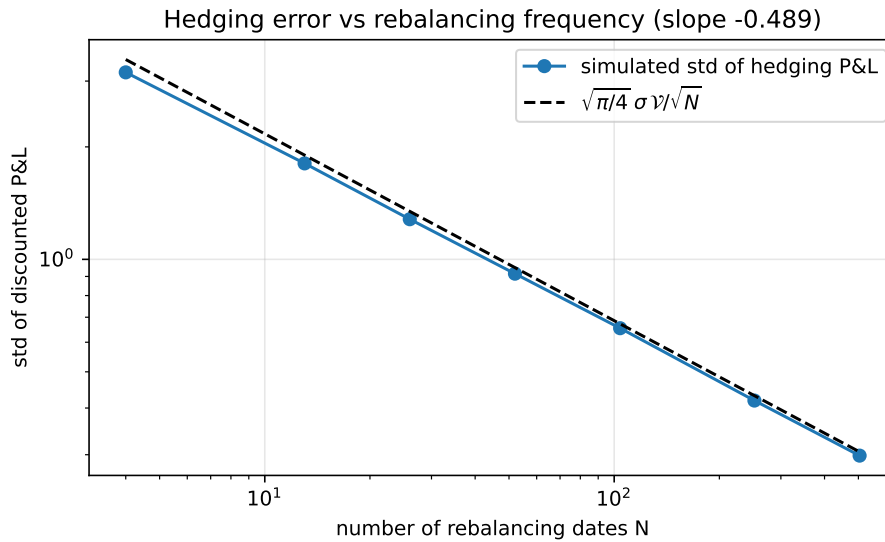


FIGURE 2.3: Standard deviation of the discounted P&L of a delta-hedged short call.

N	4	13	26	52	104	252	504
Simulated sd	3.17	1.81	1.28	0.92	0.65	0.42	0.30
Approximation (1.4)	3.43	1.90	1.34	0.95	0.67	0.43	0.31
Mean P&L	-0.06	-0.01	0.00	0.00	0.00	0.00	0.00

TABLE 2.3: Hedging P&L against the number of rebalancing dates, for a premium of 9.41.

The fitted log-log slope is -0.489 . The approximation overestimates the dispersion by about 8% at $N = 4$ and by 3% or less from $N = 26$ onwards, which is consistent with its derivation for small steps. The mean P&L is zero within noise: with correct volatility, discrete hedging adds risk but no expected cost. In practice transaction costs grow linearly with N while the risk falls only as $N^{-1/2}$, which is why desks do not rebalance continuously.

2.6 SELLING VOLATILITY TOO CHEAPLY

The option is now sold at $\sigma_{\text{imp}} = 20\%$ while the stock realises $\sigma_{\text{real}} = 25\%$, with daily rebalancing. Two hedges are compared: delta computed with the implied volatility, and delta computed with the realised one.

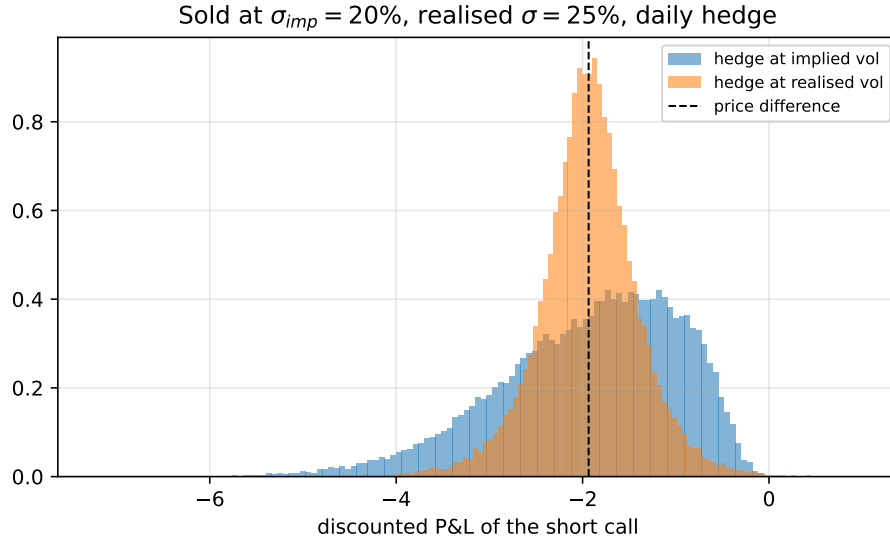


FIGURE 2.4: Distribution of the P&L of a call sold 5 volatility points too cheap.

Hedge ratio computed with	Mean	Std	5% quantile	95% quantile
implied vol (20%)	-1.928	0.975	-3.71	-0.60
realised vol (25%)	-1.936	0.531	-2.81	-1.08
$C(\sigma_{imp}) - C(\sigma_{real})$	-1.935			

TABLE 2.4: Hedging P&L under volatility misspecification (50 000 paths).

Both hedges lose the same amount on average, and that amount is the price difference $C(20\%) - C(25\%) = -1.935$. The choice of hedge volatility changes the shape of the loss, not its expectation.

With the realised volatility the hedge replicates the option that should have been sold, so the loss is locked in at inception and the only remaining noise is the discretisation error. Equation (1.4) evaluated at 25% predicts a standard deviation of 0.54 for daily rebalancing; the simulation gives 0.53. With the implied volatility the loss accumulates as $\frac{1}{2} \int \Gamma_t S_t^2 (\sigma_{imp}^2 - \sigma_{real}^2) dt$, which depends on the path: it is large when the stock stays near the strike, where gamma is high, and small when it moves away. That path dependence doubles the dispersion.

The practical reading is that a volatility trader does not need to know the future realised volatility to be right on average, but needs it to control the variance of the outcome. Of course the realised volatility is not known in advance; the second hedge is an idealised benchmark.

3

LIMITS AND EXTENSIONS

The Black–Scholes model is used here because it can be solved exactly, not because it describes markets. Three limits bound the conclusions.

- **Constant volatility.** The hedging study assumes that the stock follows a GBM with known realised volatility. Real returns have stochastic volatility and jumps, both of which add a component of hedging error that does not vanish as N grows.
- **No transaction costs.** The frequency study shows the risk side of the trade-off only. Adding proportional costs would produce an optimal rebalancing frequency, as in Leland’s model.
- **One dimension.** The advantage of the PDE is specific to a single state variable.

The natural next steps are to calibrate a Heston model to an implied volatility surface and repeat the hedging experiment when the hedger uses Black–Scholes deltas in a Heston world, and to add transaction costs in order to recover an optimal rebalancing rule.

REFERENCES

- [1] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, J. Political Economy 81 (1973).
- [2] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer (2003).
- [3] R. Rannacher, *Finite element solution of diffusion problems with irregular data*, Numer. Math. 43 (1984).
- [4] D. J. Duffy, *Finite Difference Methods in Financial Engineering*, Wiley (2006).
- [5] E. Derman and M. Kamal, *When You Cannot Hedge Continuously: The Corrections of Black–Scholes*, Risk 12 (1999).
- [6] H. E. Leland, *Option Pricing and Replication with Transactions Costs*, J. Finance 40 (1985).

APPENDIX: KEY FUNCTIONS

The full code, the tests and the experiment script are in the repository. The functions below are the core of the method.

CONTROL-VARIATE ESTIMATOR

```

46 def mc_control_variate(S0, K, T, r, sigma, n, rng, kind="call"):
47     z = rng.standard_normal(n)
48     ST = _terminal(S0, T, r, sigma, z)
49     disc = np.exp(-r * T)
50     x = disc * _payoff(ST, K, kind)
51     y = disc * ST # E[y] = S0 exactly under the risk-neutral measure
52     c = np.cov(x, y)
53     beta = c[0, 1] / c[1, 1]
54     adj = x - beta * (y - S0)
55     return MCRresult(adj.mean(), adj.std(ddof=1) / np.sqrt(n), n)

```

DISCRETE DELTA HEDGE

```

12 def hedge_pnl(S0, K, T, r, mu, sigma_true, sigma_implied, sigma_hedge,
13               n_steps, n_paths, rng):
14     dt = T / n_steps
15     S = np.full(n_paths, float(S0))
16     delta = bs_delta(S, K, T, r, sigma_hedge)
17     cash = bs_price(S0, K, T, r, sigma_implied) - delta * S
18     for k in range(1, n_steps + 1):
19         z = rng.standard_normal(n_paths)
20         S = S * np.exp((mu - 0.5 * sigma_true**2) * dt + sigma_true * np.sqrt(dt) * z)
21         cash = cash * np.exp(r * dt)
22         if k < n_steps:
23             new_delta = bs_delta(S, K, T - k * dt, r, sigma_hedge)
24             cash -= (new_delta - delta) * S
25             delta = new_delta
26     value = cash + delta * S - np.maximum(S - K, 0.0)
27     return np.exp(-r * T) * value

```